

Failure-Based Testing for Deep Reinforcement Learning Agents

WEIBIN LIN, Beihang University, China

JIANGTAO MENG, Beihang University, China

ZHENG ZHENG*, Beihang University, China

Deep Reinforcement Learning (DRL) agents have been widely adopted across diverse domains to address challenging decision-making problems, such as autonomous driving and robotic control. Given that many of these applications are safety- and security-critical, rigorous testing of DRL agents is indispensable. Existing testing methods are typically guided by reward signals to detect failures. However, for well-trained agents, whose performance approaches optimal levels in standard operating conditions, reward signals remain generally high, making current methods ineffective at uncovering critical failures.

To address these challenges, we propose a novel failure-based method that leverages task-induced failure insights to enhance failure detection capability while reducing the number of tests required. Since DRL agents are inherently designed with human-defined tasks, they provide valuable cues about task difficulty. Intuitively, a DRL agent is more likely to fail when confronted with a more difficult task; therefore, PRT prioritizes these tasks. Building on this foundation, we propose Prior Random Testing, a black-box failure-based testing method that enables targeted prioritization while preserving the diversity of generated test cases. Guided by task-induced failure insights, PRT prioritizes failure-prone regions of the input domain, thereby facilitating efficient failure detection.

PRT is evaluated on four widely used benchmarks and compared with different state-of-the-art methods including fuzzing, search-based and generative-based methods. PRT ranks among the top performers in terms of both the cost of finding the first failure and the diversity of test cases. Notably, compared to random testing, PRT achieves better diversity and reduces the testing cost by over 50%.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: Software testing, failure-based testing, deep reinforcement learning

ACM Reference Format:

Weibin Lin, Jiangtao Meng, and Zheng Zheng. 2026. Failure-Based Testing for Deep Reinforcement Learning Agents. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE178 (July 2026), 22 pages. <https://doi.org/10.1145/3808185>

1 Introduction

Artificial intelligence (AI) has shown impressive performance in solving sequential decision-making tasks using deep reinforcement learning (DRL) techniques. DRL agents are widely employed in complex tasks such as autonomous driving [17], robot control [28], traffic control [40], and gaming [26, 32]. Despite these advances, the reliability of DRL agents remains uncertain [41]. Wrong decisions made by agents could lead to catastrophic consequences, particularly in safety-critical

*Corresponding author

Authors' Contact Information: Weibin Lin, Beihang University, Beijing, China, linwb@buaa.edu.cn; Jiangtao Meng, Beihang University, Beijing, China, mengjiangtao@buaa.edu.cn; Zheng Zheng, Beihang University, Beijing, China, Zhengz@buaa.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE178

<https://doi.org/10.1145/3808185>

domains [10]. This growing reliance on DRL agents underscores the urgent need to rigorously test these agents to ensure their quality.

Real-world incidents highlight this necessity. For instance, a Tesla autonomous taxi was recently involved in its first recorded accident, colliding with a parked car [1]. Although commercial products are generally released by well-trained individuals, even rare failures can be disastrous. Therefore, efficiently testing almost flawless DRL agents is particularly important.

Existing methods that can be applied to test different types of DRL agents are typically guided by reward signals [16, 24, 29]. Reward-based guidance exploits the property that low rewards are often correlated with failures. However, when agents demonstrate well-trained performance, rewards are consistently high, offering little useful signal. For example, in the Cart Pole environment [3], all successful test cases output exactly the same reward, providing no distinction for guiding testing.

When reward signals provide limited guidance, failure-prone regions of the input domain can often be identified more directly. Since DRL agents are inherently task-driven and defined by human-specified oracles, it is possible to distinguish between easier and more difficult tasks, where the latter are more likely to induce failures. We define this intuition as task-induced failure insights, referring to the identification of input domain locations corresponding to the most difficult tasks within a given environment. A natural concern, however, is whether testing only the most difficult tasks suffices. Eniser et al. [12] show that an agent may succeed in a difficult task yet fail in an easier one. This observation underscores broad exploration: when the location of failure-prone regions is unknown a priori, evenly distributed test cases over the input space become an effective surrogate strategy for failure discovery, serving as a principled fallback when testing the most difficult tasks fails to expose failures.

This naturally connects to the idea of failure-based testing, first introduced by Chen et al. [6]. Failure-based methods select test cases based on prior knowledge of failure patterns, including geometric shapes, sizes, or locations of failure regions [6]. Adaptive Random Testing (ART) [7] is a representative example that assumes that failure-causing test cases form contiguous regions, while passing test cases also exhibit clustering behavior. Therefore, given a set of previously executed test cases that have not exposed failures, selecting new test cases farther from these existing ones increases the likelihood of discovering failures [6]. While preserving this sparsity-driven exploration strategy, we further leverage failure location information to guide failure-based testing.

Building on this, we propose Priority Random Testing (PRT), a black-box failure-based test case generation method that enables prioritization with uniformity. Before testing, PRT needs to design the hyperparameters in three steps: (1) **Failure-prone Region**: For every dimension of the input domain, identify the failure-prone value for prior testing, here corresponding to the most difficult tasks. (2) **Mapping $\mathcal{F}$** : Design a mapping $\mathcal{F}(\cdot)$ for every dimension, mapping the boundary to the failure-prone value. (3) **Confidence λ** : Define the certainty level for that failure-prone value. A larger value of λ indicates a higher prior confidence in the failure-prone designation.

PRT analytically generates new test cases within the sparsest regions of existing test cases to ensure uniformity, naturally resulting in a default priority on boundary exploration. For example, in one-dimensional input domain, regardless of the location of the first point, the farthest point from it (i.e., the sparsest location) necessarily lies at the boundary.

We briefly introduce the workflow of PRT as follows. Assume that we already have a test case set $C = \{\varepsilon_1, \varepsilon_2, \dots, \varepsilon_n\}$. When generating a new test case ε_{n+1} , PRT decomposes the problem across dimensions and generates values dimension by dimension. It first computes the largest interval E_i of each dimension i in the set C . It then selects the sparsest dimension $k = \arg \max_i (E_i)$ and generates the corresponding solution ε_{n+1}^k (the optimal solution for the k -dimensional largest interval E_k). We term this process *Dimension Reduction*. Next, PRT filters existing test cases to a

set $C' = \{x \in C \mid x^k \text{ close to } \varepsilon_{n+1}^k\}$. This step, is referred to as *Local Recombination*. By alternating *Dimension Reduction* and *Local Recombination*, a new test case ε_{n+1} is obtained. In addition, we incorporate a mapping $\mathcal{F}$ to shift boundary regions into other areas of the domain. Though PRT prioritizes testing the boundary by default, mapping $\mathcal{F}$ enables customized prioritization of other regions according to different failure insights. We implement PRT with a time complexity of approximately $O(MN^2)$ for generating N test cases in an M -dimensional domain.

In summary, this paper makes the following contributions:

- We propose a failure-based method, PRT, with a time complexity $O(MN^2)$, capable of leveraging the task-induced failure insights on DRL agents to efficiently find failures.
- We evaluate PRT against four state-of-the-art (SOTA) methods—including fuzzing, search-based, and generative-based methods—on three representative testing subjects with distinct reward structures. PRT consistently ranks among the top performers in terms of both effectiveness and efficiency for detecting the first failure while achieving the best diversity of test cases.
- We further empirically investigate the failure patterns of different DRL agents to evaluate the effectiveness and generalizability of PRT. Our findings reveal two main types of patterns: block-shaped and point-shaped. PRT addresses these through task-induced failure insights and uniform distribution, respectively.

2 Background

2.1 Preliminaries

Markov Decision Process (MDP). MDP is a mathematical framework used for modeling decision-making, which is composed of a tuple $\langle S, A, \mathcal{T}, \mathcal{R}, \pi \rangle$. S is a set of all possible states during MDP. A is a set of actions available to the decision-making agent. $\mathcal{T}(s_{t+1}|s_t, a_t) \rightarrow [0, 1]$ defines the probability from the present state s_t to the next state for every $s_{t+1} \in S$ with action a_t . The reward function $\mathcal{R}(s_t, a_t) \rightarrow \mathbb{R}$ is used to evaluate the quality of an action a_t under a specified state s_t . Specifically, an episode reward means the cumulative reward from the starting state s_0 to the end of the episode. $\pi(a_t|s_t) \rightarrow [0, 1]$ gives the probability of taking action a_t under the state s_t , representing the behavior of an agent.

Deep Reinforcement Learning (DRL). Based on the mathematical MDP model, DRL gives a solution of $\pi(a_t|s_t)$ [2]. Unlike deep learning, which directly labels an action as right or wrong, RL only scores an action with a reward value and lets the agents learn how to obtain the cumulative reward in a task as high as possible. The environment in RL refers to the tuple $\langle S, A, \mathcal{T}, \mathcal{R} \rangle$, and the agent refers to the policy π . The environment is responsible for giving the observed state and the corresponding reward to the agent, and the agent gives an action to the environment for the next state transition. As the deep neural network (DNN) demonstrates excellent performance in many areas, DRL employs DNN as the policy function $\pi(a_t|s_t)$. Popular DRL techniques include DQN [37], PPO [31], TQC [19], and so on.

Test Cases for DRL. Though our testing subject, the DRL agent, takes states as inputs, our test cases are defined as the initial configurations of the environment. Once the agent takes the first action, we will not change anything, so as to ensure the authenticity of the scenario.

Input Domain. The input domain is defined as the set of all feasible test cases. It is emphasized that the test case is not the input s for testing subjects.

Test Oracle. We give a setting of the environment as the test case at the beginning. Then the agent interacts with the environment. If the state s_t satisfies some conditions (depended on the environment), the test case fails or passes at the t step.

Failure Pattern. We refer to the failure pattern as the characteristics of the failure test case distribution in the input domain, especially the geometric ones after visualization.

2.2 Task-Induced Failure Insights on DRL Agents

A concern is whether task-induced failure insights can generalize across different DRL agents. We demonstrate that such insights are broadly applicable because they originate from properties shared by all DRL agents. Specifically, every DRL agent is designed to solve a man-made task, and these tasks inherently contain cues about difficulty [34]. The core intuition is that agents are more likely to fail in more difficult tasks, with difficulty intuited directly from the task definition. For example, in games, difficulty can be increased by adding more obstacles or enemies, while in control problems, test cases can be designed farther from the stable state.

Amal et al. make a comprehensive review on testing DRL agents and summarizes the testing environments [33]. We select the environments tested by more than two papers and annotate their tasks along with the failure insights on the failure patterns in Table. 1, so as to show that we can easily locate the failure-prone regions of the input domain. We also attach the cited numbers counted by Amal et al. [33].

Table 1. Task-induced insights about failures in different environments [33]

Environment	Task	Insights ¹	Papers
Frozen Lake	Walk from start to goal without falling into any holes	More holes	3
Mountain Car ²	Accelerate a car from the bottom of a sinusoidal valley to the top of the right hill	Starting from the middle with no velocity	8
Cart Pole ²	Balance a pole attached by an un-actuated joint to a cart	Higher initial velocity	12
Lunar Lander ²	Safely land on the middle pad from the sky	Higher initial velocity	7
Super Mario	Arrive at the destination without collision with the enemies or falling into the pits	More enemies and pits	3
Coop Navi (Simple Spread)	Several agents need to cover the same number of landmarks while avoiding collisions	Agents are closer to each other	3
Bipedal Walker	Balance a 4-joint robot to step over obstacles	Larger size of obstacles	4
CARLA	Control a car to arrive at the destination without collision	More vehicles on roads	4
Taxi	Navigate a car to pick up a passenger and drop him off at the destination	The locations are far from each other	3

¹ Insights indicate how to design a test case with a higher failure probability.

² For these three environments, we introduce more details in Sec. 4.2.

3 Approach

PRT consists of two cooperative mechanisms: dimension reduction and local recombination. Dimensionality reduction provides a global view that identifies sparse regions among the candidate set, while local recombination examines which region is still dense and offer that as the new candidate set. Each time PRT generates a new test case ε , it performs dimension reduction by splitting the test case space along individual dimensions: $\varepsilon = (\varepsilon^1, \varepsilon^2, \dots, \varepsilon^m)$. Each time, PRT selects the sparsest dimension (assumed to be the k_1 -th dimension) from the candidate set C to generate the corresponding optimal solution *value* so that $\varepsilon^{k_1} = \text{value}$. Then PRT reconstructs a candidate set $C_1 \subset C$ by only including the test cases in C close to ε^{k_1} in the k_1 -th dimension (local recombination). Next round we will have ε^{k_2} assigned to the *value* corresponding to the sparsest k_2 -th dimension, and a smaller candidate set $C_2 \subset C_1$, and so on up to k_m , where $\{k_1, k_2, \dots, k_m\} = \{1, 2, \dots, m\}$. At last we can obtain a new test case ε .

$$FP(InD) = \{fp^1, fp^2, \dots, fp^m\} \quad (1)$$

By default, PRT prioritizes testing the boundary. However, the region we want to prioritize (corresponding to the most difficult tasks) does not always lie in the boundary. At the beginning of applying PRT, we need to identify the failure-prone (FP) region of the input domain (InD) as Eq. 1. More specifically, for every dimension of the input domain InD^i , we need to specify the failure-prone value fp^i . By default, the value is set as the boundary, both the left side and the right side. We can change PRT's exploring strategy by a well-designed mapping $\mathcal{F}$, shifting the prior testing region. We regard $\mathcal{F}$ to be a hyperparameter as it is manually defined before testing. After setting the other hyperparameters, we can begin testing with PRT.

3.1 Approach Overview

Alg. 1 shows the workflow of PRT. It takes the number of test cases N and test case dimension M as inputs along with hyperparameters $\mathcal{F}, \lambda \in [1, +\infty)$, and gives the generated test case set $\mathbb{T}$ as the output. Line 2 randomly generates the first test case ε as the initial reference. Line 3 constructs the output test case set $\mathbb{T}$. The outer **while** loop, lines 4–17, generates a new test case ε at each round, where line 16 adds it to the output test case set $\mathbb{T}$. Line 5 creates a candidate set, initialized as the existing test case set $\mathbb{T}$. Line 6 creates an empty container for the generating test case ε and Line 7 assigns an adaptive hyperparameter for the inner **while** loop. Line 8 creates an index set for indicating which dimensional value of ε has not been generated. The inner **while** loop, lines 9–15, generates a dimensional value for ε at each round. Each round line 10 picks the sparsest dimension k (i.e., with the largest interval) from the candidate set C and generates the corresponding value resulting from the largest interval, which is assigned to $\Lambda[k]$ -dimensional of ε (i.e., $\varepsilon^{\Lambda[k]}$) at line 11. Line 12 uses such information to select test cases similar to $\varepsilon^{\Lambda[k]}$, where the new candidate set C is a subset of the old one. As we have generated k -dimensional data, line 13 reduces that dimension of C and the index set Λ removes that dimension (e.g., remove the second dimension of $\{(1, 2, 3), (4, 5, 6)\} \rightarrow \{(1, 3), (4, 6)\}$). Until all dimensional data have been generated, we can obtain a new test case ε . Lines 18–20 make a mapping $\mathcal{F}$ on every test case in $\mathbb{T}$.

Algorithm 1 Prior Random Testing

Input: expected number of test cases N , test case dimension M , mapping $\mathcal{F}$, confidence λ

Output: test case set $\mathbb{T}$

```

1: function PRT( $N, M, \mathcal{F}, \lambda$ )
2:    $\varepsilon_{random} \leftarrow$  randomly generate an  $M$ -dimensional test case
3:    $\mathbb{T} \leftarrow \{\varepsilon_{random}\}$ 
4:   while  $|\mathbb{T}| < N$  do ▷  $|\mathbb{T}|$ : the number of  $\mathbb{T}$ 
5:      $C \leftarrow copy(\mathbb{T})$ 
6:      $\varepsilon \leftarrow$  an  $M$ -dimensional empty array ▷ the next test case
7:      $\tau \leftarrow (|\mathbb{T}| + 1)^{-1/M}$  ▷ an adaptive hyperparameter for LOCALRECOMBINATION
8:      $\Lambda \leftarrow \{1, 2, \dots, M\}$  ▷ a dimensional index set
9:     while  $\Lambda$  is not empty do
10:       $k, optimal\_solution \leftarrow DIMENSIONREDUCTION(C, M, \lambda)$ 
11:       $\varepsilon^{\Lambda[k]} \leftarrow optimal\_solution$  ▷  $\Lambda[k]$ : the  $k$ -th element of  $\Lambda$ 
12:       $C \leftarrow LOCALRECOMBINATION(C, k, optimal\_solution, \tau)$ 
13:      delete  $k$ -dimensional data of  $C$ 
14:       $\Lambda \leftarrow \Lambda \setminus \{k\}$  ▷ remove  $k$  from  $\Lambda$ 
15:    end while
16:     $\mathbb{T} \leftarrow \mathbb{T} \cup \{\varepsilon\}$ 
17:  end while
18:  for  $k = 1 \rightarrow |\mathbb{T}|$  do
19:     $\mathbb{T}[k] \leftarrow \mathcal{F}(\mathbb{T}[k])$ 
20:  end for
21:  return  $\mathbb{T}$ 
22: end function

```

3.2 Dimension Reduction

As PRT generates a new test case dimension by dimension, dimension reduction compares the largest intervals of every dimension in the candidate set C , generates the corresponding value. More specifically, given a M -dimensional test case set $C = \{\varepsilon_1, \varepsilon_2, \dots, \varepsilon_n\}$, the largest interval of i -th dimension is defined as E_i :

$$\begin{aligned} & \max_{x \in \text{InD}^i} E_i \\ & \text{s.t.} \quad E_i \leq |x - \varepsilon_k^i|, \quad k = 1, 2, \dots, n \end{aligned} \quad (2)$$

where $1 \leq i \leq M$, InD^i is the i -dimensional input domain, ε_k^i is the i -dimensional value of ε_k . And the optimal solution x is referred to the corresponding solution for E_i .

Idea. Given n points in high-dimensional domain, finding the farthest point from them is an NP-hard (Nondeterministic Polynomial time) problem. However, the problem is easy to be solved in one-dimensional domain.

For example, assuming the input domain is $[0, 1]$, we have four existing points (colored blue): $p_1 = 0, p_2 = 0.1, p_3 = 0.3, p_4 = 0.7$. We can compute their interval as Eq. 3.

$$(p_2, p_3, p_4) - (p_1, p_2, p_3) = (0.1, 0.2, 0.4) \quad (3)$$

It is easy to find a large interval between p_3 and p_4 . The corresponding solution is $p = (p_3 + p_4)/2 = 0.5$ and the interval is $(p_4 - p_3)/2 = 0.2$. However, by taking the infimum and the supremum into account, we find that $p = 1$ is the optimal solution because of its larger interval $1 - p_4 = 0.3 > (p_4 - p_3)/2 = 0.2$. So the new generated point for this round is $p = 1$ (colored green) and the maximal interval is 0.3. Similarly, we can get the next generated point $p = 0.5$ (colored orange) and its corresponding interval 0.2.

Furthermore, the sparsest point is treated as a probabilistic rather than deterministic solution, as it may not always be the optimal choice. We design a probabilistic function as Eq. 4, where $\inf$ and $\sup$ are the infimum and the supremum respectively, $\lambda \geq 1$ is a hyperparameter, and $\theta \in [0, 1]$ follows a uniform distribution. Eq. 4 makes the generated point more likely close to $(\inf + \sup)/2$ and λ controls the probability. The larger λ is, the more likely $f(\theta | \inf, \sup, \lambda)$ is close to $(\inf + \sup)/2$. When $\lambda = 1$, $f(\theta | \inf, \sup, \lambda)$ follows a uniform distribution.

$$f(\theta | \inf, \sup, \lambda) = \begin{cases} \frac{\inf + \sup}{2} - (1 - 2\theta)^\lambda \cdot \frac{\sup - \inf}{2}, & 0 \leq \theta \leq 0.5 \\ \frac{\inf + \sup}{2} + (2\theta - 1)^\lambda \cdot \frac{\sup - \inf}{2}, & 0.5 < \theta \leq 1 \end{cases} \quad (4)$$

Here $(\inf + \sup)/2$ is the middle point. As θ follows a uniform distribution $U(0, 1)$, the generated point moves left when $\theta < 0.5$, and moves right when $\theta > 0.5$. $|1 - 2\theta|^\lambda$ is an item ranging from 0 to 1. The larger λ , the smaller $|1 - 2\theta|^\lambda$ and the more likely $f(\theta | \inf, \sup, \lambda)$ close to the middle point. We refer λ as the confidence level. It is correlated with the probability function of new generating points. More specifically, in the case of left boundary, θ is required to follow a uniform distribution $U(0.5, 1)$. The larger λ , the more likely new points close to the boundary. And so like for the right boundary.

Design. Algo. 2 formulates the workflow to find the sparsest dimension and the corresponding value. Lines 5–30 record the maximal interval and the corresponding solution dimension by dimension. Lines 7–8 aim to only consider the boundary (lines 18–27). Lines 10–12 compute the intervals, and lines 13–14 record the largest one. Line 15 samples a value from the uniform distribution between 0 and 1. Line 16 computes the generated value according to Eq. 4. Lines 18–27 take a similar method to deal with the boundary case, where $\inf^i$ and $\sup^i$ are the infimum and the supremum of i -dimensional input domain. Lines 28–29 record the analytical result of the i -th dimension after normalization. Line 31 selects the dimension with the maximal interval. Line 32 obtains the solution by the dimension number.

Algorithm 2 Dimension Reduction**Input:** test case set C , dimension M , confidence λ **Output:** the sparsest dimension k , the corresponding solution *value*

```

1: function DIMENSIONREDUCTION( $C, M, \lambda$ )
2:    $c_1, c_2, \dots, c_n \leftarrow$  all test cases in  $C$ 
3:    $intervals \leftarrow \emptyset$ 
4:    $solutions \leftarrow \emptyset$ 
5:   for  $i = 1 \rightarrow M$  do
6:      $p_1, p_2, \dots, p_n \leftarrow \text{sort}(c_1^i, c_2^i, \dots, c_n^i)$  ▷ in ascending order
7:     if  $n == 1$  then
8:        $max\_interval \leftarrow 0$ 
9:     else
10:      for  $j = 1 \rightarrow n - 1$  do
11:         $d_j \leftarrow p_{j+1} - p_j$ 
12:      end for
13:       $k \leftarrow \arg \max_j (d_j)$ 
14:       $max\_interval \leftarrow (p_{k+1} - p_k) / 2$ 
15:       $\theta \leftarrow \text{UniformSample}(0, 1)$  ▷ Sample a value from a uniform distribution  $[0, 1]$ 
16:       $solution \leftarrow f(\theta \mid p_k, p_{k+1}, \lambda)$ 
17:    end if
18:    if  $max\_interval < p_1 - inf^i$  then ▷  $inf^i$ : the infimum of the  $i$ -th dimension
19:       $max\_interval \leftarrow p_1 - inf^i$ 
20:       $\theta \leftarrow \text{UniformSample}(0.5, 1)$ 
21:       $solution \leftarrow f(\theta \mid 2 \cdot inf^i - p_1, p_1, \lambda)$ 
22:    end if
23:    if  $max\_interval < sup^i - p_n$  then ▷  $sup^i$ : the supremum of the  $i$ -th dimension
24:       $max\_interval \leftarrow sup^i - p_n$ 
25:       $\theta \leftarrow \text{UniformSample}(0, 0.5)$ 
26:       $solution \leftarrow f(\theta \mid p_n, 2 \cdot sup^i - p_n, \lambda)$ 
27:    end if
28:     $intervals \leftarrow intervals \cup \{max\_interval / (sup^i - inf^i)\}$ 
29:     $solutions \leftarrow solutions \cup \{solution\}$ 
30:  end for
31:   $k \leftarrow \arg \max(intervals)$ 
32:   $value \leftarrow solutions[k]$ 
33:  return  $k, value$ 
34: end function

```

3.3 Local Recombination

Idea. Algo. 2 aims to identify the largest interval containing a given number of points across different dimensions. However, this task becomes challenging when the number of points is large and the dimensionality is high. Moreover, Algo. 2 fails to preserve the relationships among test cases across dimensions. Therefore, we aim to reconstruct the candidate set C . And we regard that the generation process should focus on points close to the newly generated *solution*.

Design. Eq. 5 defines the local recombination strategy, where $k_1, k_2, \dots, k_m$ are the generated dimensions of ε , Q_τ denotes τ -quantile.

$$C' = \text{Recombine}(\mathbb{T} \mid \varepsilon, \tau) = \bigcap_{i=1}^m \left\{ x \in \mathbb{T} \mid |x^{k_i} - \varepsilon^{k_i}| \leq Q_\tau \left(\{ |y^{k_i} - \varepsilon^{k_i}| : y \in \mathbb{T} \} \right) \right\} \quad (5)$$

If k_{m+1} is newly generated dimension and C is the last recombination candidate set, Eq. 5 can be simplified as only computing the k_{m+1} -th dimension:

$$C' = \text{Recombine}(C \mid \varepsilon, \tau) = \left\{ x \in C \mid |x^{k_{m+1}} - \varepsilon^{k_{m+1}}| \leq Q_\tau \left(\{ |y^{k_{m+1}} - \varepsilon^{k_{m+1}}| : y \in C \} \right) \right\} \quad (6)$$

Algorithm 3 Local Recombination**Input:** test case set C , target dimension number k and new value $solution$, resolution τ **Output:** the recombined test case set C'

```

1: function LOCALRECOMBINATION( $C, k, solution, \tau$ )
2:    $Dist \leftarrow \emptyset$ 
3:   for  $c \in C$  do
4:      $Dist \leftarrow Dist \cup \{|c^k - solution|\}$ 
5:   end for
6:    $q_\tau \leftarrow Q_\tau(Dist)$ 
7:    $C' \leftarrow \{c \in C \mid |c^k - solution| \leq q_\tau\}$ 
8:   return  $C'$ 
9: end function

```

Algo. 3 formulates the local recombination process. Line 6 obtains the τ -quantile of the distance set. A simple method is to sort the points and obtain the $\tau|Dist|$ -th point, whose time complexity is $O(n \log n)$. Here we leverage Introsselect algorithm to reduce the time complexity to $O(n)$.

There is a question on how to select the hyperparameter τ . From Algo. 3, we can get $|C| = \lceil \tau|C|' \rceil \approx \tau|C|'$. Initially in the inner **while** loop of Algo. 1, there are $|\mathbb{T}|$ points and the iteration will have M times. By designing $N \cdot \tau^M = 1$, we have a recommended setting:

$$\tau = N^{-\frac{1}{M}} \quad (7)$$

3.4 Boundary Priority

In this section, we explain why PRT prioritizes boundary regions by default.

First, Algo. 2 assigns a boundary interval that is twice as large as that of a middle region, prioritizing generating boundary points. For example, suppose the input domain is $[0, 1]$ and the existing points are 0.3, 0.8. The interval corresponding to the candidate solution at 0.55 is only 0.25, whereas the interval at the boundary point 0 is 0.3. Thus, the next generated point will be 0.

Moreover, Algo. 3 further reinforces boundary exploration across dimensions. For instance, assume the existing points lie on the boundary in the i -th dimension but in the middle region of the j -th dimension. Although the new point may be generated in the middle region along the i -th dimension, it will be positioned on the boundary along the j -th dimension.

3.5 Linear Mapping

We demonstrate that PRT generates evenly distributed test cases but prioritizes testing the boundary by default ($\mathcal{F}(t) = t$). However, sometimes we may expect PRT to prioritize other regions. Here we give an example of prioritizing the middle domain. Eq. 8 assumes t is a scalar value with infimum inf and supremum sup and maps the boundary to the middle $(inf + sup)/2$. For high-dimensional data, this process is repeated for every dimension.

$$\mathcal{F}_1(t) = \begin{cases} \frac{3inf+sup}{2} - t, & inf \leq t \leq \frac{inf+sup}{2} \\ \frac{inf+3sup}{2} - t, & \frac{inf+sup}{2} < t \leq sup \end{cases} \quad (8)$$

Note that we make a linear transformation to ensure the evenly distributed property. Other transformations can also be adopted for specific purposes.

3.6 Realization

If we follow the steps of Alg. 2 and Alg. 3, the time complexity is $O(MN^2 \log N)$. We use some data structure techniques to accelerate the program.

Assume that we want to generate N M -dimensional test cases and we already have k test cases during the **while** loop in Alg. 1. First, an $M \times N$ matrix D is used to store test cases. i -th row records the i -dimensional data in an ascending order, despite the relationship between different dimensions. That makes the time complexity of Alg. 2 line 6 $O(1)$ and that of the whole Alg. 2 $O(Mk)$. Secondly, use a $M \times k$ matrix α to record which number of test case $D(i, j)$ is from, and use a $k \times M$ matrix β to record which index of D the k -th test case is in. They satisfy $\alpha(m, \beta(j, m)) = j, \forall 1 \leq j \leq k, 1 \leq m \leq M$. That makes the time complexity of the whole Alg. 3 $O(\tau k(M-1))$. At the next iteration, the number of test cases approximate τk and the time complexity of Algo. 2 and Algo. 3 will be $O(\tau k(M-1))$ and $O(\tau^2 k(M-2))$ respectively. As the time complexity of Algo. 3 is lower than that of Algo. 2, the time complexity of Lines 8–14 in Algo. 1 is $O(\tau^t k(M-t))$, where k and t implies the k -th outer **while** loop and the t -th inner **while** loop. By traversing k from 1 to N we can get the result $O(MN^2)$.

$$\sum_{k=2}^N \sum_{t=0}^{M-1} O(\tau^t k(M-t)) \Big|_{\tau=k^{-\frac{1}{M}}} \approx \sum_{k=2}^N O(Mk) = O(MN^2) \quad (9)$$

4 Experimental Setup

4.1 Baselines

PRT is a black-box setting method and can be applied to test any type of DRL agents. Our baselines include SOTA fuzzing methods MDPFuzz [29] and CureFuzz [16], search-based method QD [24], generative-based method G-Model [20] as well as random testing (RT). For QD, we use the Map-Elites algorithmic version as it performs the best in most cases analyzed by the authors [24]. Note that though these methods are all black-box, most of them need preparation before testing. Besides, the information used for testing also varies.

Table 2. Applicable conditions of different methods.

	Failure Insights	Execution	Model Training	State	Action	Reward
PRT (ours)	✓	×	×	×	×	×
MDPFuzz [29]	×	✓	✓	✓	×	✓
CureFuzz [16]	×	✓	✓	✓	×	✓
G-Model [20]	×	×	✓	×	×	×
QD [24]	×	✓	×	✓	✓	✓
RT	×	×	×	×	×	×

Tab. 2 summarizes their differences. PRT needs failure insights about the task. MDPFuzz, CureFuzz and QD need to execute a number of test cases before testing. MDPFuzz, CureFuzz and G-Model will train machine learning models before testing. Furthermore, these methods access different information during testing. MDPFuzz and CureFuzz require the executing data, including state and reward, while QD requires state, action and reward.

For MDPFuzz [29] and CureFuzz [16], which require executing test cases before testing, we randomly generate 2000 passing test cases, where we choose 2000 to give the method enough information and the passing condition is their inner algorithmic step. Though QD [24] needs execution before testing, it incorporates the execution into the testing cost.

4.2 Testing Subjects

We select the three most popular environments in Tab. 1 and one high-dimensional environment for testing, which are all developed by OpenAI in Gymnasium platform [36]. The three most popular environments are also chosen because of the different reward patterns (RQ1 in Sec. 5.2, Fig. 2). We introduce the environments' tasks, the test case definitions and the test oracles as follows. To comprehensively test different types of agents, we select the discrete version of Cart Pole and the continuous versions of Lunar Lander and Mountain Car.

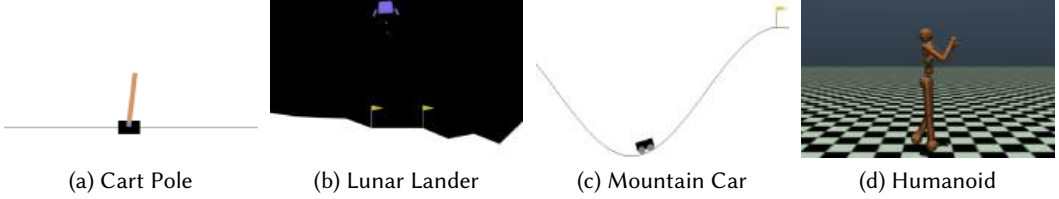


Fig. 1. Different environment examples

4.2.1 Cart Pole. This environment corresponds to the version of the cart-pole problem described by Barto et al. [3]. In Fig. 1a, a pole is attached by an unactuated joint to a cart, which moves along a frictionless track. The test case is defined as its four initial states $(x, \dot{x}, \theta, \dot{\theta})$: the cart's position $x \in [-0.9, 0.9]$, the cart's velocity $\dot{x} \in [-0.3, 0.3]$, the pole's angle $\theta \in [-\frac{\pi}{60}, \frac{\pi}{60}]$ and the pole's angular velocity $\dot{\theta} \in [-\frac{\pi}{90}, \frac{\pi}{90}]$. The agent can apply either a left or a right force to the cart. Once the pole's angle is not within $\pm 12^\circ$ or the cart's position is not within ± 2.4 , the task fails. If the agent fails at t step, it will have a reward t . If the system keeps balance during 500 steps, we will terminate testing, give a reward 500 and regard the test case as passing.

4.2.2 Lunar Lander. This is a classic rocket trajectory optimization problem. In Fig. 1b, a lander descends from the middle sky towards a designated landing pad on the lunar surface. The test case is defined as the initial force (f_x, f_y) applied to the lander along x direction $f_x \in [-1500, 1500]$ and y direction $f_y \in [-1500, 1500]$. The agent can make one of four discrete actions: do nothing, fire the left orientation engine, fire the right orientation engine, or fire the main engine. A task fails if the lander crashes (e.g., outside the landing pad or with excessive speed), while it succeeds if the lander safely lands on the middle pad. The reward is calculated according to the distance to the landing pad, the speed, the angle, and the leg contacts. A successful landing within the designated area yields a high reward of 100, while crashing results in a large negative reward of -100.

4.2.3 Mountain Car. This environment first appeared in Andrew Moore's PhD thesis (1990) [27]. In Fig. 1c, a car is situated between two hills and must build momentum to reach the flag positioned at the top of the right hill. The test case is defined by its two initial states (x, v) : the car's horizontal position $x \in [-1, 0]$ and velocity $v \in [-0.05, 0.05]$, where the position of the valley is $-\frac{\pi}{6} \approx -0.52$ and the position of the flag is 0.5. The agent can apply one of three discrete actions: accelerate left, accelerate right, or do nothing. The agent receives a constant reward of -1 per step until termination. If the car reaches the flag at time step t , the task succeeds with a cumulative reward of $-t$; but if $t > 200$, the test case is regarded as a failure.

4.2.4 Humanoid-v4. This environment is a 3D bipedal robot simulation based on the work of Tassa et al. [35]. In Fig. 1d, a humanoid robot with a torso, a pair of arms and legs (each leg has three parts, each arm has two parts) is designed to simulate human locomotion. The robot starts in a standing pose, and the test case is defined as the bias added to the standing pose. The input domain

is $[-0.01, 0.01]$ ⁴⁷, including 24 position values and 23 velocity values. The agent controls the robot by applying torques to its 17 hinge joints, including 3 for the abdomen, 3 for each hips, 1 for each knees, 2 for each arm's shoulder, and 1 for each elbow, totally $3 + (3 + 1 + 2 + 1) \times 2 = 17$. The goal is to make the robot walk forward as fast as possible without falling. The task fails if the robot falls (the torso height drops below a threshold). At each time step, the agent receives a reward comprised of a forward velocity bonus, a healthy reward for staying upright, and penalties for excessive control forces. If the agent survives until the time horizon (1000 steps), the test case is regarded as passing.

Validity of test cases. We regard that a test case ε is valid if and only if there exists a feasible action sequence enabling the agent to pass the test case. We have verified that the most difficult test cases (i.e., the most difficult tasks of specific environments) within the designed input domain can successfully be solved, which implies all failures are caused by incorrect software logic (i.e., incorrect agent actions) rather than flaws in the input domain design. In addition, as we change the initial state bound of Cart Pole, Lunar Lander and Mountain Car, we upload three oracle agents to our open-source repository for verifying that all failures can be solved.

Task-induced failure insights. In the Cart Pole environment (Fig. 1a), the agent must balance the cart and the pole with $(x, \dot{x}, \theta, \dot{\theta})$ around $(0, 0, 0, 0)$, so the most difficult task corresponds to the boundary $(\pm 0.9, \pm 0.3, \pm \frac{\pi}{60}, \pm \frac{\pi}{90})$. In the Lunar Lander environment (Fig. 1b), the lander is required to safely land on the middle pad, so the most difficult task also corresponds to the boundary $(\pm 1500, \pm 1500)$. In the Mountain Car environment (Fig. 1c), the car must climb the right hill to reach the flag within specific steps, so the most difficult task corresponds to the middle point $(-0.5, 0)$. In the Humanoid environment (Fig. 1d), the robot needs to maintain the standing pose, so the most difficult task corresponds to the boundary $(\pm 0.01, \pm 0.01, \dots, \pm 0.01)$.

4.3 Hyperparameters

Mapping $\mathcal{F}$. For Cart Pole, Lunar Lander and Humanoid, the mapping $\mathcal{F}$ does not change anything $\mathcal{F}(t) = t$. For the Mountain Car environment, we shift the boundary to the middle with Eq. 8 $\mathcal{F}_1$.

Confidence λ . We set $\lambda = 20$. Under this setting, assuming the parameter θ follows a uniform distribution $\theta \sim U(0, 1)$, the deviation term $(1 - 2\theta)^\lambda$ in Eq. 4 exceeds 10% with probability 10.9%, and exceeds 2% with probability 17.8%.

4.4 Implementation

Stochasticity of test case execution. Stochasticity is a common challenge in testing DRL agents. To minimize its impact on the outputs, we generate test cases using different random seeds but execute all test cases under a fixed random seed, which means the test case generation is non-deterministic but the test case execution is deterministic.

Experimental platform. Our experimental platform is 24.04.1-Ubuntu, with hardwares consisting of an AMD Ryzen 9 9950x CPU and two 32 GB 5200 Hz RAMs. We do not use any GPU for testing.

5 Evaluation

In this section, we present three research questions (RQs) to study the effectiveness, efficiency and generalization of PRT.

RQ1: How effective and efficient is PRT for detecting failures?

We evaluate PRT against SOTA fuzzing, search-based, and generative methods on DRL agents with failure rates below 1% measured by random testing under 100,000 tests. The comparison focuses on the testing cost required to detect the first failure. In addition, we examine the conditions under which PRT outperforms the baselines, as well as those in which the baselines demonstrate superior performance.

RQ2: How diverse are test cases generated by different methods?

As we claim that PRT preserves the diversity of generated test cases while enabling prioritization, RQ2 evaluates the extent of this diversity, particularly in comparison with the baselines. In addition, we visualize the distributions of all generated test cases to provide further insights into the strengths and limitations of different algorithms beyond scalar diversity metrics.

RQ3: What are the failure patterns of different agents, and how does PRT address them?

As a failure-based testing method, the effectiveness of PRT is strongly correlated with the failure pattern of testing subjects. To this end, we train agents with different algorithms and analyze their failure patterns. We then investigate why PRT can effectively detect failures across different DRL agents.

5.1 Diversity Metric

We quantify the diversity of generated test cases using differential **entropy**, which measures the dispersion of samples in continuous state spaces. Given a continuous random variable $X \in \mathbb{R}^d$ with probability density function $p(x)$, its differential entropy is defined as:

$$H(X) = - \int p(x) \log p(x) dx \quad (10)$$

Differential entropy characterizes the effective volume occupied by a distribution and increases as samples become more dispersed within the same support [8]. Under identical dimensionality and normalization, higher entropy indicates broader exploration of the state space rather than closeness to a specific target distribution.

Since the underlying distribution of test cases is unknown and may exhibit complex, non-Gaussian structures, we adopt a non-parametric k-nearest neighbor (kNN) entropy estimator. Specifically, we use the estimator proposed by Kozachenko and Leonenko [18]. Given a set of n samples $x_{i=1}^n \subset \mathbb{R}^d$, the entropy is estimated as:

$$\hat{H}_k = \psi(n) - \psi(k) + \log V_d + \frac{d}{n} \sum_{i=1}^n \log \varepsilon_k(x_i) \quad (11)$$

where $\psi(\cdot)$ denotes the digamma function, V_d is the volume of the unit ball in $\mathbb{R}^d$, and $\varepsilon_k(x_i)$ is twice the Euclidean distance from x_i to its k -th nearest neighbor. Unless otherwise stated, we fix the neighborhood size to $k = 10$ in all experiments, following common practice in non-parametric entropy estimation [4, 14, 18].

To ensure meaningful comparison, all entropy values are computed under identical state dimensionality, normalization, and neighborhood size, and are compared only within the same environment. Under these controlled conditions, the estimated entropy serves as a relative measure of sample dispersion, reflecting the extent to which an algorithm explores the continuous state space.

5.2 RQ1: Effectiveness and Efficiency for Detecting Failures

At the beginning, we construct some well-trained agents for testing. We employ PPO [31] to train Cart Pole and Lunar Lander agents, TQC [19] to train Mountain Car agents, and DSAC-T [11] to train Humanoid agents. We repeat training and random testing for 100,000 times until two conditions are both satisfied: (1) there is at least one failure test case under 100,000 tests, and (2) the failures are less than 500. Specifically, the numbers of failures under 100,000 tests are 226, 164, 56, 112 for Cart Pole, Lunar Lander, Mountain Car and Humanoid respectively. We use the F-measure (the expected number of tests required to detect the first failure), proposed by Chen et al. [7], as an effectiveness metric, as it directly reflects the failure-detection capability of a testing algorithm.

Table 3. Testing cost for finding the first failure with corresponding (p -value, effect size)

	Cart Pole		Lunar Lander		Mountain Car		Humanoid	
	Number	Time	Number	Time	Number	Time	Number	Time
PRT	62.65	1.85	70.15	1.60	444.3	2.20	524.9	165.9
MDPFuzz	2341.4 (0.00,0.98)	465.7 (0.00,1.00)	616.1 (0.00,0.92)	92.7 (0.00,0.98)	96.0 (1.00,0.19)	4.68 (0.00,0.79)	1268.0 ¹ (0.03,0.68)	400.1 ¹ (0.02,0.69)
CureFuzz	2182.6 (0.00,0.93)	68.4 (0.00,0.93)	613.5 (0.00,0.91)	15.1 (0.00,0.92)	1502.4 (0.00,0.80)	8.42 (0.00,0.82)	1116.6 (0.21,0.58)	352.4 (0.20,0.58)
G-Model	657.5 (0.00,0.81)	24.5 (0.00,0.83)	836.3 (0.02,0.69)	24.2 (0.01,0.71)	96192+ ² (0.00,1.00)	968+ ² (0.00,1.00)	4008.2 (0.00,0.94)	1310 (0.00,0.94)
QD	1616.5 (0.00,0.89)	48.6 (0.00,0.89)	566.9 (0.00,0.88)	13.2 (0.00,0.88)	656.1 (0.01,0.74)	4.23 (0.00,0.78)	1789.3 (0.17,0.59)	572.3 (0.14,0.60)
RT	491.5 (0.00,0.89)	15.2 (0.00,0.90)	532.8 (0.00,0.89)	12.7 (0.00,0.89)	2177.1 (0.00,0.79)	10.26 (0.00,0.79)	1261.3 (0.01,0.75)	393.2 (0.01,0.75)

The item ranking first is colored red, while the second is colored blue. The statistics with p -value larger than 5% is colored orange. Number and Time mean the number of generated test cases and the corresponding time (in seconds, including algorithmic computation and environment simulation) for finding the first failure.

- 1 In the Humanoid environment, MDPFuzz requires approximately 553 seconds to analyze a single test case, implying that preparing 2,000 seeds would take more than 12 days before testing can even begin. Due to this prohibitive computational cost, we disable the freshness-guided component of MDPFuzz in the Humanoid environment.
- 2 Across 20 independent experiments, G-Model detected a failure only once, occurring at the 23,851st test case. In the remaining 19 experiments, testing was terminated after 100,000 trials without identifying any failures. Consequently, the minimum cost required for G-Model to detect a failure is $(23,851 + 100,000 \times 19) / 20 = 96,192$ in terms of test executions and 968 in terms of accumulated time cost.

Under the low failure rate condition, we compare the testing cost of different methods to find the first failure test case. We consider the testing cost from two aspects: the number of generated test cases and the total testing time. For simulated programs, we usually want to find failures as fast as possible. However, for some practical applications, we may consider the cost of executing a test case, where the number of test cases is more important. For each experiment, we repeat it 20 times and report the average cost together with the p -value and the effect size. For each experiment, we repeat the evaluation 20 times and report the average cost, together with the corresponding p -value and effect size. We employ Mann-Whitney U test to compute the p -value, under the one-sided assumption that the cost of the baselines is *lower* than that of PRT. The effect size between the baselines and PRT is measured using the $\hat{A}_{21}$ statistic [38].

Tab. 3 reports the average testing cost required by different methods to detect the first failure across various environments. In terms of average cost, PRT achieves the best performance in all cases, except for the number cost in Mountain Car. On one hand, the failure insights towards Mountain Car are deviated, where the failures are not precisely located in the central region. On the other hand, the reward pattern has strong guidance in this environment and MDPFuzz makes an aggressive strategy. Further analysis will be provided later in Fig. 2 and Fig. 3. Nevertheless, compared with RT, PRT consistently reduces the testing cost by at least 40%, requiring significantly fewer test cases to detect the first failure. Moreover, PRT incurs low computational overhead, achieving the shortest time cost across all testing subjects. In Mountain Car, although the number cost of PRT is nearly six times that of MDPFuzz, it still detects the first failure faster.

Tab. 3 also reports the corresponding p -values and effect sizes. Overall, PRT performs statistically better than RT across all environments. However, in some cases, the advantage of PRT is not statistically significant, particularly in the Humanoid environment. On one hand, the low average cost achieved by PRT can be attributed to its smaller maximal cost. In Humanoid, PRT's maximal number cost is 1461, whereas RT, MDPFuzz, CureFuzz, and QD exhibit maximal number costs of 3069, 5135, 7858, and 7992, respectively. As will be shown in RQ2, PRT achieves the highest diversity, which may contribute to its stability. On the other hand, the statistical insignificance can be partly attributed to the high variance and limited repetitions in the experiments. The results indicate that, under limited experimental runs, some baselines can achieve competitive performance with PRT in specific scenarios, especially in the high-dimensional Humanoid environment.

Focusing on the average cost, we observe that MDPFuzz achieves the best number cost in Mountain Car. Notice that MDPFuzz, CureFuzz, and QD all benefit from reward guidance (primarily, though not exclusively, driven by reward signals). To better understand the underlying mechanisms, we further analyze the reward patterns.

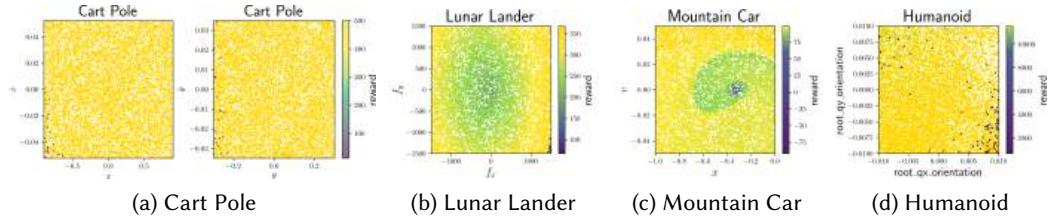


Fig. 2. Reward patterns of different environments

We draw the heat map as Fig. 2. For Humanoid, we execute 100,000 random test cases and select 10,000 test cases with the lowest rewards. For the other environments, we execute 10,000 random test cases. As the input domain of Cart Pole is four-dimensional, we present two plots (more details in Sec. 4.2). For Humanoid, since the input domain has 47 dimensions, we enumerate all $\binom{47}{2} = 1081$ dimension pairs and select the pair with the lowest diversity among **failure test cases**, as computed by Eq. 11, which implies failures are clustered most at that projections. The failure test cases achieve the lowest rewards and are colored dark purple. More specifically, the failure test cases of Cart Pole (Fig. 2a) are on the left side, those of Lunar Lander (Fig. 2b) appear in the lower-right region, those of Mountain Car (Fig. 2c) are concentrated near the center, close to $(-0.3, 0)$, and those of Humanoid (Fig. 2d) are mainly distributed in the lower-right and the upper-left regions.

We observe that in Cart Pole (Fig. 2a), the rewards of all passing test cases are identical, meaning the reward signal offers no guidance for distinguishing between passing and failing cases. In this setting, PRT and RT achieve the best performance. In Lunar Lander (Fig. 2b), although the rewards vary across the input domain, the passing test cases in the central region yield low rewards, which may mislead methods guided by reward signals. In this case, reward-driven baselines perform much better than those in Cart Pole, yet still fall short of PRT. In Mountain Car (Fig. 2c), passing test cases exhibit diverse rewards. Fortunately, the failure region is surrounded by areas of low reward, allowing MDPFuzz and QD to efficiently detect failures. Nevertheless, PRT consistently ranks second in terms of the number cost among all methods. In Humanoid (Fig. 2d), part of the failure test cases are located near the medium-reward test cases (green points), particularly in the upper-left region. However, these failure test cases are also surrounded by high-reward ones. As a result, MDPFuzz and CureFuzz can detect failures quickly in some cases, but in others it requires significantly more testing cost to detect failures.

Answer to RQ1: Compared with RT, PRT reduces the testing cost by at least 50% across all testing subjects. Compared with DRL-based testing techniques, PRT's advantage is particularly evident in the environments where reward signals are either uninformative or misleading.

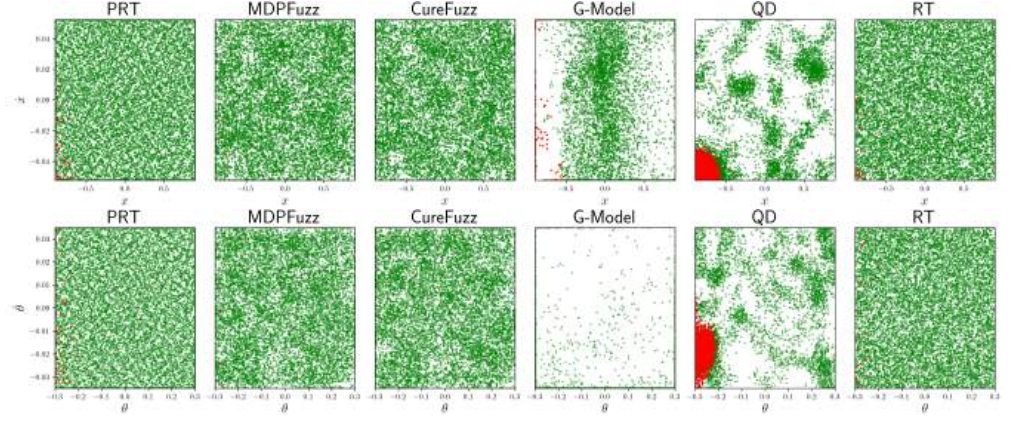
5.3 RQ2: Diversity of Test Cases

We leverage entropy (Eq. 11) to evaluate the diversity of generated test cases by different methods. Each method is required to generate 10,000 test cases, and every experiment is repeated 20 times. To mitigate the threat of conclusion validity, we show the results with hyperparameters $k = 5$, $k = 10$ and $k = 20$ in Eq. 11. We also report the standard variance of each experiment. Beyond such distance metric, we also visualize the test case distribution of different methods as Fig. 3. On one hand, this can implement the distance metric, making our conclusion more reliable. On the other hand, more importantly, that visualization intuitively shows the exploring strategies of different methods, enabling deeper analysis on their advantages and disadvantages. Note that, during the 20 experiments, we select the one with the lowest number cost finding the first failure for visualization, where the green and the red points represent the passing and the failure test cases respectively.

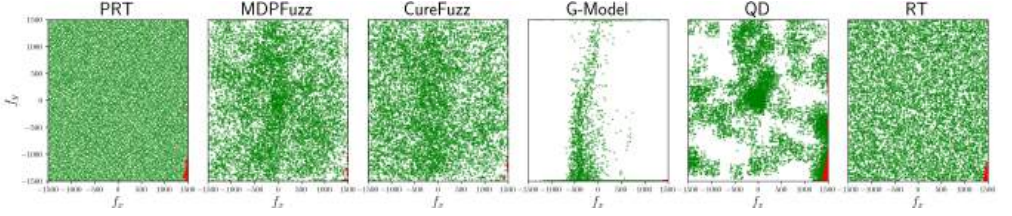
Table 4. Entropy of 10,000 generated test cases under different k

Method	k	Cart Pole	Lunar Lander	Mountain Car	Humanoid
PRT		0.5266 $\pm$ 0.0043	0.2306 $\pm$ 0.0014	0.2120 $\pm$ 0.0018	14.3975 $\pm$ 0.4199
MDPFuzz	5	-0.6569 $\pm$ 0.0205	-0.0446 $\pm$ 0.0092	-0.5768 $\pm$ 0.0184	-40.4884 $\pm$ 0.6664
CureFuzz		-0.6567 $\pm$ 0.0203	-0.0289 $\pm$ 0.0129	-0.0320 $\pm$ 0.0135	-31.1420 $\pm$ 0.6763
G-Model		-22.6375 $\pm$ 2.6734	-9.6081 $\pm$ 0.7587	-10.0524 $\pm$ 1.0541	-59.9535 $\pm$ 2.8108
QD		-4.4818 $\pm$ 0.4454	-1.2159 $\pm$ 0.1458	-1.2140 $\pm$ 0.1607	-83.1634 $\pm$ 0.7219
RT		0.1765 $\pm$ 0.0068	0.0157 $\pm$ 0.0053	0.0157 $\pm$ 0.0053	14.2386 $\pm$ 0.0387
PRT		0.4513 $\pm$ 0.0028	0.1054 $\pm$ 0.0015	0.0880 $\pm$ 0.0019	14.9516 $\pm$ 0.3636
MDPFuzz	10	-0.3097 $\pm$ 0.0210	-0.0360 $\pm$ 0.0082	-0.5727 $\pm$ 0.0176	2.3939 $\pm$ 0.5797
CureFuzz		-0.3093 $\pm$ 0.0203	-0.0104 $\pm$ 0.0082	-0.0148 $\pm$ 0.0102	11.6711 $\pm$ 0.3869
G-Model		-22.0423 $\pm$ 2.7388	-9.0635 $\pm$ 0.7885	-9.5217 $\pm$ 1.0936	-52.3097 $\pm$ 2.8589
QD		-4.3464 $\pm$ 0.4261	-1.1395 $\pm$ 0.1427	-1.1994 $\pm$ 0.1615	-80.6062 $\pm$ 1.2369
RT		0.2142 $\pm$ 0.0071	0.0215 $\pm$ 0.0057	0.0215 $\pm$ 0.0057	14.6885 $\pm$ 0.0383
PRT		0.4044 $\pm$ 0.0027	0.0911 $\pm$ 0.0009	0.0731 $\pm$ 0.0010	15.5194 $\pm$ 0.3137
MDPFuzz	20	-0.0061 $\pm$ 0.0187	-0.0292 $\pm$ 0.0077	-0.5719 $\pm$ 0.0177	14.6547 $\pm$ 0.1209
CureFuzz		-0.0049 $\pm$ 0.0177	0.0012 $\pm$ 0.0083	-0.0039 $\pm$ 0.0073	14.8709 $\pm$ 0.0826
G-Model		-21.2772 $\pm$ 2.7295	-8.5240 $\pm$ 0.8100	-9.0049 $\pm$ 1.1442	-42.7788 $\pm$ 2.9618
QD		-4.1132 $\pm$ 0.3829	-1.0618 $\pm$ 0.1529	-1.1918 $\pm$ 0.1622	-77.3323 $\pm$ 1.8267
RT		0.2589 $\pm$ 0.0069	0.0290 $\pm$ 0.0037	0.0290 $\pm$ 0.0037	15.1581 $\pm$ 0.0371

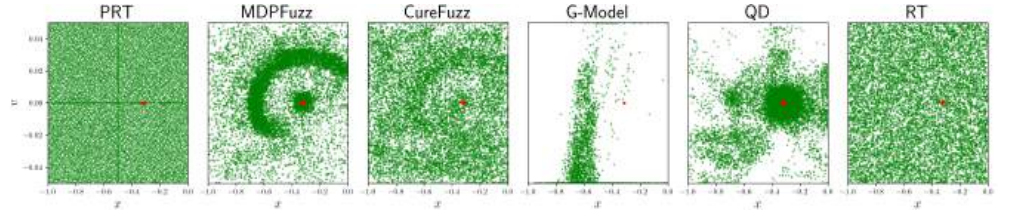
From Tab. 4, PRT consistently achieves the highest diversity across all environments, indicating that the test cases generated by PRT are more well-dispersed and uniformly distributed in the input space. This observation is further supported by the visualizations in Fig. 3b and Fig. 3c, where PRT produces a more evenly distributed set of test cases than RT. In the other high-dimensional environments, though diversity of PRT is high in Tab. 4, Fig. 3 cannot reveal the dimensional combinations, resulting in little difference on the two-dimensional prejections. Moreover, as shown



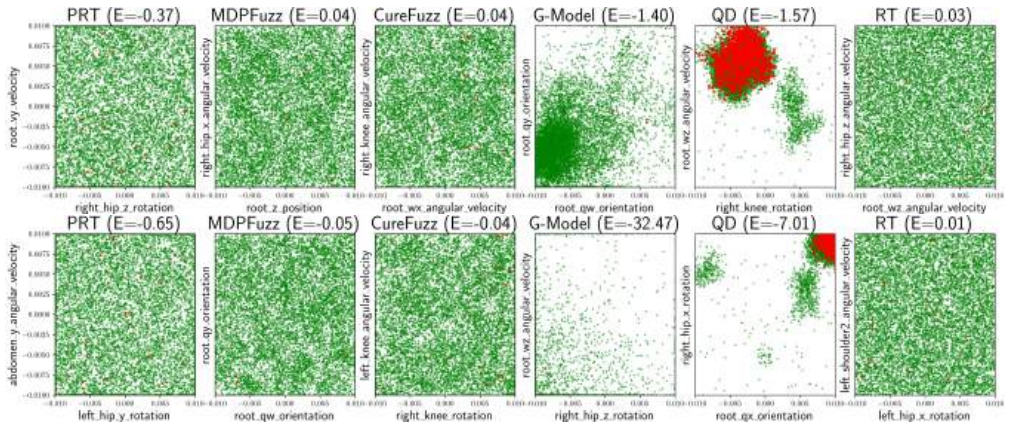
(a) Cart Pole



(b) Lunar Lander



(c) Mountain Car



(d) Humanoid

Fig. 3. Test case distributions of different methods

in Fig. 3c, PRT tends to focus more on the middle region of the input domain due to the mapping defined in Eq. 8 in the Mountain Car environment. However, since the failure region in Mountain Car deviates from the center of the input space, this bias reduces the efficiency of PRT in detecting the first failure, reported in Tab. 3.

For Humanoid, since the input domain has 47 dimensions, we enumerate all $\binom{47}{2} = 1081$ dimension pairs and select the two pairs with the highest and lowest entropy among **10,000 test cases**, as reported in Fig. 3d. From Fig. 3, we can observe that different algorithms exhibit distinct preferences. MDPFuzz seems to be more interested in the middle region in Lunar Lander (Fig. 3b) and the circle region in Mountain Car (Fig. 3c), which fit the reward pattern shown in Fig. 2b and Fig. 2c. It corresponds to the best number cost in Tab. 3 Mountain Car environment. CureFuzz achieves the third-highest diversity, following PRT and RT. G-Model exhibits very low diversity. He et al. observed that G-Model tends to concentrate on boundary regions [16]. Moreover, a GitHub [issue](#) reported that the generative model fails to converge during training, a problem we also encountered, leading to test cases outside the input domain. Following the solution proposed by Li et al., we project invalid test cases onto the boundary of the input domain. QD demonstrates a low diversity but finds many failures. That is because QD randomly samples a cluster of test cases but always selects the one with the lowest reward for executing. Upon detecting a failure, QD concentrates on mutating the failure test case.

Both G-Model and QD take an aggressive strategy—they focus on specific domains with a large number of test cases—but they may miss some failures. G-Model miss failures in Mountain Car. QD does not find the failures around $(x, \dot{x}) = (-0.9, 0.0)$ in Cart Pole environment (the middle left domain of the first line). Moreover, even if we add more test cases, it is still difficult for these methods to find the failures in such domain. That highlights the importance of test case diversity, which can help us miss fewer failures in testing.

Answer to RQ2: PRT achieves the best diversity of test cases among all methods. Notably, PRT consistently performs better than RT across all testing subjects.

5.4 RQ3: Failure Patterns

In this section, we examine the failure patterns exhibited by different agents. Specifically, we empirically study the regions of the input domain where failures occur. Chen et al classified the patterns of failure test cases into three categories: *point*, *strip* and *block* patterns [7]. However, the dimensionality of Humanoid input domain is 47. Limited number of evenly distributed test cases cannot reveal the failures' geometric characteristics. Even if we only place two points in every dimension, the combinations can be $2^{47} \approx 10^{11}$. So we just explore failure patterns of the other three environments. For Cart Pole (discrete version), we select agents trained with A2C [25], QRDN [9], and PPO [31]; for Lunar Lander (continuous version), SAC [15], TD3 [13], and PPO [31]; and for Mountain Car (continuous version), DDPG [21], TRPO [30], and TQC [19]. In total, eight algorithms are considered to ensure the robustness of our conclusions. Testing is performed using PRT, which achieves the highest diversity according to Tab. 4. For each testing subject, PRT generates up to 100,000 test cases to reduce the likelihood of missing failures.

Note that PRT is characterized by: (1) prioritizing specific regions (boundary in Cart Pole, Lunar Lander and Humanoid), and (2) uniform distribution. That implies PRT is significantly effective for failures aligned with the manually defined regions and *block-shaped* failures.

Fig. 4 illustrates the distribution of passing test cases (green) and failure test cases (red). We find that the testing subjects exhibit *block-shaped* (clustered) failures aligned with the task-induced failure insights. That is the reason why PRT achieve significant effectiveness in Tab. 3. Moreover,

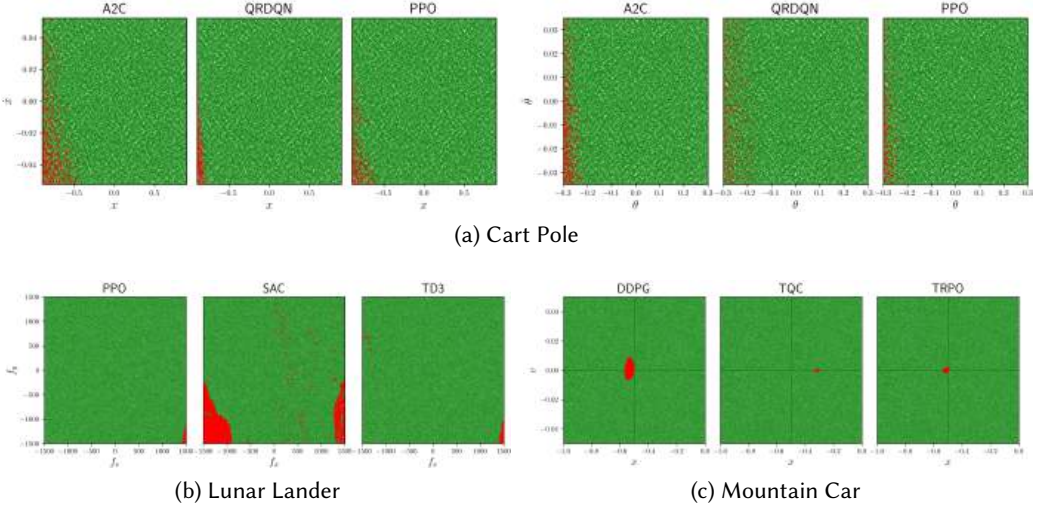


Fig. 4. Failure patterns of different agents.

we observe that such aligned and *block-shaped* patterns appear across different training algorithms in the same environment, which implies the task-induced failure insights are largely environment-dependent and agent-independent and such failure insight is a general tool for testing DRL agents. However, there are also *point-shaped* (dispersed) failures not aligned with the failure insights. Though not efficiently, PRT detects them by uniform distribution.

Answer to RQ3: We empirically observe that there are *block-shaped* failures aligned with the task-induced failure insights across different training algorithms in the same environment, where PRT detects such failures by testing priority. There are also *point-shaped* failures not aligned with the task-induced failure insights. PRT can detect them by uniform distribution.

6 Discussion

The applicable conditions of PRT. Though PRT is capable of generating evenly distributed points within a hypercube (defined as Eq. 12), certain techniques can be applied to broaden its applicability.

$$\mathcal{D} = \{x \in \mathbb{R}^m | x^i \in [\inf_i, \sup_i], i = 1, 2, \dots, m\} \quad (12)$$

A hypercube is essentially a domain in which each dimension is independently bounded. To satisfy the boundedness assumption, a sufficiently large value can be chosen to approximate the infimum or the supremum. For the independence assumption, if the domain has a complex shape, a larger hypercube can be used to enclose the domain. Points generated outside the target domain are filtered out, while those within are retained.

How to understand the hyperparameters τ and λ ? PRT assumes that the failure region is a hypercube with side length τ , and it seeks to prevent two test cases from residing within the same τ -length hypercube. Nevertheless, owing to randomness, it is challenging to strictly determine whether two points lie within such a region. Hence, assuming the existing points are evenly

distributed, we employ a ratio model as an approximation. As PRT prioritizes testing a specific domain, λ exhibits confidence in that task-induced failure insight. If $\lambda = 1$, the generating process follows a uniform distribution and that means the task-induced failure insight plays little role in the test case generation process.

Can PRT be applied to test other software systems? The answer is yes. PRT is a failure-based testing method. We use PRT to test DRL agents because we can easily locate their failure-prone input domains. If we know about the size or the potential failure domain of other softwares, we can also leverage PRT to test them.

Why some methods perform worse than RT in Tab. 3? Firstly, the evaluation metrics differ: while most methods are assessed by the total number of failures detected, we measure the cost for finding the first failure. Secondly, the methods have different search preferences. Many are effective at uncovering clustered failures, whereas RT distributes its effort across the entire input domain, avoiding excessive testing in any local region. At last, we have explained that in some environments they lack effective reward guidance, which will make their performance worse.

7 Threats to Validity

Internal Validity. A potential threat to internal validity lies in the setting of hyperparameters. For PRT, we describe the hyperparameters in Sec. 4.3. For the baselines, we strictly follow the settings reported by their authors whenever available. For MDPFuzz [29], Mazouni et al. confirmed that the hyperparameters are set as $K = 10$, $\tau = \gamma = 0.01$ [23]. For CureFuzz [16], the authors did not report the hyperparameter settings. We therefore examined its [GitHub repository](#) and found that the settings differ from the formulas in the paper. In our experiments, we followed the GitHub settings, normalized the rewards of different environments to $[0, 1]$, and set $\alpha = 5$, $\beta = 0.01$, $\gamma = 1$ for all experiments. For G-Model [20], all hyperparameters are clearly stated in the paper, and we adopt them directly. For QD [24], we refer to its [GitHub repository](#). As it also uses the Lunar Lander environment, we take the same settings for that task. For Cart Pole, Mountain Car and Humanoid, we derive the settings by analogy from those of Taxi.

External Validity. A limited number of testing subjects poses a threat to external validity. One concern is how to design task-induced failure insights for other environments. As discussed in Sec. 2.2, such insights can be derived from the tasks, which are intrinsic characteristics of DRL agents. Another concern is the generalizability of our findings. To mitigate this threat, we analyze the underlying mechanisms of different testing methods from multiple perspectives (e.g., reward patterns, test case distributions, and failure patterns), with the aim of explaining why the observed behaviors are expected, in order to generalize beyond the studied benchmarks.

Construct Validity. A threat to construct validity lies in the diversity metric. Eq. 11 can be unstable due to large dimension d or different hyperparameter k . However, d only influences the constant item, and that is equivalent when we only compare the entropy under the same environment and the same test case number. To mitigate the threat from k , we report $k = 5, 10, 20$ in RQ2.

Conclusion Validity. A threat to conclusion validity may come from the randomness of test execution. To mitigate this, we repeat each experiment 20 times and report the averaged results. In addition, we report (p -value, effect size) in RQ1, and standard variance in RQ2. To further improve reproducibility, we fix the random seed in our open-source implementation.

8 Related Works

Fuzz testing. Pang et al. proposed MDPFuzz [29], which leverages rewards to calculate seed sampling probabilities and employs a Gaussian Mixture Model (GMM) to estimate state freshness.

Mazouni et al. verified that while the GMM strategy is effective, it is also time-consuming [23]. He et al. developed CureFuzz [16], a curiosity-driven fuzzing approach. Wan et al. introduced DRLFuzz [39], a white-box fuzzing method that uses a coverage-guided algorithm and the Q-network gradient to mutate seeds for testing DRL agents.

Search-based Testing. Zolfagharian et al. proposed a white-box search-based testing method STARLA [42]. It leverages the Q-value of agents to encode the state sequence into a binary sequence and applies a random forest algorithm to predict the failure probability of that sequence as the fitness function. Biagiola and Tonella introduced Indago [5], searching for failure environment configurations with a failure predictor to give the failure probability. However, both STARLA and Indago rely on training data—test cases and their corresponding passing or failure labels. Mazouni et al. characterize a sequence with a 2-dimensional array as the agents' behavior, so as to guide genetic search for failures [24]. Ma et al. proposed MASTest [22], a diversity-guided testing method for multi-agent systems.

Generative-based Testing. Li et al. introduced the diffusion model to generate test cases [20]. In every epoch, random test cases are sampled to fine-tune the diffusion model, which then generates a batch of new test cases for evaluation.

9 Conclusion and Future Work

In this work, we focus on well-trained DRL agents and propose a failure-based testing technique PRT. Our key point is that DRL agents are task-oriented and we can intuitively have the failure insights about which task is harder and the corresponding region has a higher failure probability. With the task-induced failure insights, we leverage PRT to prioritize testing the specific regions of the input domain, in order to uncover failures as fast as possible. Our empirical experiments show that PRT can effectively and efficiently find failures in terms of the number of test cases and the test execution time. Its advantage is particularly evident in the environments where reward signals are either uninformative or misleading for reward-guided algorithms. Furthermore, we investigate the failure patterns of several agents across different environments and we find that the task-induced failure insights are empirically aligned with the block-shaped failures while the characteristic of uniform distribution can help PRT to find the point-shaped failures.

In the future, we plan to apply PRT to testing other DRL agents according to their specific failure insights as well as other types of software. For example, the failure-prone regions of codes can be the boundary or branch conditions in the program, which can be served as the failure-insights for PRT to test. Besides, since PRT is efficient in finding the first failure and some methods can exploit specific failure characteristics, we will further investigate the feasibility of combining PRT with these methods, aiming to improve the effectiveness of detecting clustered failures while maintaining high efficiency and uniform distribution.

Acknowledgements

This work was partially supported by the National Key R&D Program of China (Grant No. 2024YFB33 11503) and the National Natural Science Foundation of China (Grant No. 62372021). We sincerely thank Tsong Yueh Chen (Swinburne University of Technology), Yuechen Li (Beihang University), Yi Cai (Beihang University), and Tianjie Zhou (Beijing Normal University) for helpful discussions and insightful comments.

Data Availability

We open our source code at <https://github.com/Avagnes/PRT-DRL-Experiments/tree/main> to show more details and help for further researches.

References

- [1] 2025. [Tesla Robotaxi Hits Parked Car in First Recorded Accident](#). PC Magazine.
- [2] Kai Arulkumaran, Marc Peter Deisenroth, Miles Brundage, and Anil Anthony Bharath. 2017. Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine* 34, 6 (2017), 26–38.
- [3] Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. 1983. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics* SMC-13, 5 (1983), 834–846. doi:10.1109/TSMC.1983.6313077
- [4] Jan Beirlant, Edward J Dudewicz, László Györfi, Edward C Van der Meulen, et al. 1997. Nonparametric entropy estimation: An overview. *International Journal of Mathematical and Statistical Sciences* 6, 1 (1997), 17–39.
- [5] Matteo Biagiola and Paolo Tonella. 2024. Testing of Deep Reinforcement Learning Agents with Surrogate Models. *ACM Trans. Softw. Eng. Methodol.* 33, 3, Article 73 (March 2024), 33 pages. doi:10.1145/3631970
- [6] Tsong Yueh Chen, Fei-Ching Kuo, Robert G. Merkel, and T.H. Tse. 2010. Adaptive Random Testing: The ART of test case diversity. *Journal of Systems and Software* 83, 1 (2010), 60–66. doi:10.1016/j.jss.2009.02.022 SI: Top Scholars.
- [7] T. Y. Chen, H. Leung, and I. K. Mak. 2005. Adaptive Random Testing. In *Advances in Computer Science - ASIAN 2004. Higher-Level Decision Making*, Michael J. Maher (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 320–329.
- [8] Thomas M Cover and Joy A Thomas. 2006. *Elements of Information Theory*. Vol. 1. John Wiley & Sons.
- [9] Will Dabney, Mark Rowland, Marc G. Bellemare, and Rémi Munos. 2018. Distributional reinforcement learning with quantile regression. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence* (New Orleans, Louisiana, USA) (AAAI’18/IAAI’18/EAAI’18). AAAI Press, Article 353, 10 pages.
- [10] Yi Dong, Xingyu Zhao, Sen Wang, and Xiaowei Huang. 2024. Reachability Verification Based Reliability Assessment for Deep Reinforcement Learning Controlled Robotics and Autonomous Systems. *IEEE Robotics and Automation Letters* 9, 4 (2024), 3299–3306. doi:10.1109/LRA.2024.3364471
- [11] Jingliang Duan, Wenxuan Wang, Liming Xiao, Jiaxin Gao, Shengbo Eben Li, Chang Liu, Ya-Qin Zhang, Bo Cheng, and Keqiang Li. 2025. Distributional Soft Actor-Critic With Three Refinements. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47, 5 (2025), 3935–3946. doi:10.1109/TPAMI.2025.3537087
- [12] Hasan Ferit Eniser, Timo P. Gros, Valentin Wüstholtz, Jörg Hoffmann, and Maria Christakis. 2022. Metamorphic relations via relaxations: an approach to obtain oracles for action-policy testing. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual, South Korea) (ISSTA 2022). Association for Computing Machinery, New York, NY, USA, 52–63. doi:10.1145/3533767.3534392
- [13] Scott Fujimoto, Herke Hoof, and David Meger. 2018. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*. PMLR, 1587–1596.
- [14] Weihao Gao, Sreeram Kannan, Sewoong Oh, and Pramod Viswanath. 2017. Estimating Mutual Information for Discrete-Continuous Mixtures. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/ef72d53990bc4805684c9b61fa64a102-Paper.pdf
- [15] Tuomas Haarnoja, Aurick Zhou, P. Abbeel, and Sergey Levine. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *ArXiv abs/1801.01290* (2018). <https://api.semanticscholar.org/CorpusID:28202810>
- [16] Junda He, Zhou Yang, Jieke Shi, Chengran Yang, Kisub Kim, Bowen Xu, Xin Zhou, and David Lo. 2024. Curiosity-Driven Testing for Sequential Decision-Making Process. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–14.
- [17] David Isele, Reza Rahimi, Akansel Cosgun, Kaushik Subramanian, and Kikuo Fujimura. 2018. Navigating occluded intersections with autonomous vehicles using deep reinforcement learning. In *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2034–2039.
- [18] Leonid F. Kozachenko and Nikolai N. Leonenko. 1987. Sample estimate of the entropy of a random vector. *Problems of Information Transmission* 23, 2 (1987), 95–101.
- [19] Arsenii Kuznetsov, Pavel Shvechikov, Alexander Grishin, and Dmitry Vetrov. 2020. Controlling overestimation bias with truncated mixture of continuous distributional quantile critics. In *International Conference on Machine Learning*. PMLR, 5556–5566.
- [20] Zhuo Li, Xiongfei Wu, Derui Zhu, Mingfei Cheng, Siyuan Chen, Fuyuan Zhang, Xiaofei Xie, Lei Ma, and Jianjun Zhao. 2023. Generative model-based testing on decision-making policies. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 243–254.
- [21] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971* (2015).
- [22] Xuyan Ma, Yawen Wang, Junjie Wang, Xiaofei Xie, Boyu Wu, Shoubin Li, Fanjiang Xu, and Qing Wang. 2024. Enhancing multi-agent system testing with diversity-guided exploration and adaptive critical state exploitation. In *Proceedings of*

- the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1491–1503.
- [23] Quentin Mazouni, Helge Spieker, Arnaud Gotlieb, and Mathieu Acher. 2024. Policy Testing with MDPFuzz (Replicability Study). In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1567–1578.
 - [24] Quentin Mazouni, Helge Spieker, Arnaud Gotlieb, and Mathieu Acher. 2024. Testing for Fault Diversity in Reinforcement Learning. In *Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024)* (Lisbon, Portugal) (AST '24). Association for Computing Machinery, New York, NY, USA, 136–146. doi:10.1145/3644032.3644458
 - [25] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous Methods for Deep Reinforcement Learning. In *International Conference on Machine Learning*. <https://api.semanticscholar.org/CorpusID:6875312>
 - [26] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602* (2013).
 - [27] Andrew William Moore. 1990. *Efficient Memory-based Learning for Robot Control*. Technical Report. University of Cambridge.
 - [28] Hai Nguyen and Hung La. 2019. Review of deep reinforcement learning for robot manipulation. In *2019 Third IEEE international conference on robotic computing (IRC)*. IEEE, 590–595.
 - [29] Qi Pang, Yuanyuan Yuan, and Shuai Wang. 2022. Mdpfuzz: testing models solving markov decision processes. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 378–390.
 - [30] John Schulman, Sergey Levine, Philipp Moritz, Michael Jordan, and Pieter Abbeel. 2015. Trust region policy optimization. In *Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37* (Lille, France) (ICML '15). JMLR.org, 1889–1897.
 - [31] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
 - [32] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumar, Thore Graepel, et al. 2018. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* 362, 6419 (2018), 1140–1144.
 - [33] Amal Sunba, Jameleddine Hassine, and Moataz Ahmed. 2026. Testing reinforcement learning systems: A comprehensive review. *Journal of Systems and Software* 231 (2026), 112563. doi:10.1016/j.jss.2025.112563
 - [34] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. A Bradford Book, Cambridge, MA, USA.
 - [35] Yuval Tassa, Tom Erez, and Emanuel Todorov. 2012. Synthesis and stabilization of complex behaviors through online trajectory optimization. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. 4906–4913. doi:10.1109/IROS.2012.6386025
 - [36] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulao, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. 2024. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032* (2024).
 - [37] Hado Van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 30.
 - [38] András Vargha and Harold D. Delaney. 2000. A Critique and Improvement of the "CL" Common Language Effect Size Statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25, 2 (2000), 101–132. <http://www.jstor.org/stable/1165329>
 - [39] Xiaohui Wan, Tiancheng Li, Weibin Lin, Yi Cai, and Zheng Zheng. 2024. Coverage-guided fuzzing for deep reinforcement learning systems. *Journal of Systems and Software* 210 (2024), 111963.
 - [40] Cathy Wu, Abdul Rahman Kreidieh, Kanaad Parvate, Eugene Vinitzky, and Alexandre M Bayen. 2021. Flow: A modular learning framework for mixed autonomy traffic. *IEEE Transactions on Robotics* 38, 2 (2021), 1270–1286.
 - [41] Hitoshi Yoshioka and Hirotada Hashimoto. 2024. A Reliability Quantification Method for Deep Reinforcement Learning-Based Control. *Algorithms* 17, 7 (2024). doi:10.3390/a17070314
 - [42] Amirhossein Zolfagharian, Manel Abdellatif, Lionel C Briand, Mojtaba Bagherzadeh, and S Ramesh. 2023. A search-based testing approach for deep reinforcement learning agents. *IEEE Transactions on Software Engineering* 49, 7 (2023), 3715–3735.

Received 2026-02-24; accepted 2026-03-24